Grammatika: G=<V,W,S,P> ahol:
V: terminális jelek, a nyelv karakterkészletének halmaza,
W: nem terminális jelek, a grammatikai szimbólumok halmaza,
S: kezdőszimbólum, nem terminális jel
P: a helyettesítési szabályok véges halmaza,

Nyelv: A G nyelvtan által generált L(G) nyelv a G-ben elérhető terminális szavak összessége. Jelsorozatok, amik a G grammatika segítségével tetszőleges számú lépésben generálhatók.
Chomsky nyelvosztályok: A grammatikákat, az azokat meghatározó helyettesítési szabályok alapján Chomsky négy osztályokba sorolta, az osztály sorszámának növekedésével egyre szigorúbb megkötéseket írt elő:
- 0. típusú nyelvosztály: *általános grammatika*: semmiféle külön megkötést sem tartalmaz
- 1. típusú nyelvosztály: *környezetfüggő grammatika*: $\alpha A\beta \rightarrow \alpha\omega\beta$ $\alpha,\beta,\omega \in (VUW)^*$ $A \in W$ $\omega \neq \varepsilon$ *(ε: üres szó)*
- 2. típusú nyelvosztály: *környezetfüggetlen grammatika*: $A \rightarrow \omega$ $\omega \in (VUW)^*$ $A \in W$ $\omega \neq \varepsilon$
  + S nem lehet szabályok jobboldalán (ilyen elvű pl. egy C++ compiler)
- 3. típusú nyelvosztály: *reguláris grammatika*: $A \rightarrow aB$, $A \rightarrow a$ $A,B \in W$ $a \in V$
  + Ha $S \rightarrow \varepsilon$ van a szabályok közt, akkor S nem állhat a szabályok jobboldalán
  (pl. ilyen elven ellenőrzi a Word helyesírás-ellenőrzőjében a jeleket/ragokat/képzőket)

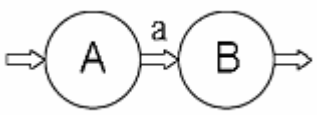
Tehát reguláris nyelvtanokban, a bal oldalon csak egy egyedülálló nemterminális szimbólum állhat, de a jobb oldalra is megkötést kell tenni: lehet üres, lehet egyetlen terminális szimbólum és lehet egy egyedül álló terminális szimbólumot követő nemterminális szimbólum.

Lexikális elemző: fő feladata egy reguláris nyelv segítségével a tokenek felismerése. (Tokenek: A programozási nyelv alapelemei (kulcsszavak, azonosítók, címkék, stb.)) Az eredeti kódból egy tömörített programot állít elő a későbbi feldolgozáshoz. Egyéb feladatai a forrásprogram formatálása, a megjegyzések kiszűrése, a fordítóprogram direktívák feldolgozása, valamint a szimbólum- és attribútumtáblák előkészítése.

Véges determinisztikus automaták: A reguláris nyelveket a 3-as (Chomsky-féle) osztályba tartozó grammatikák generálják. Az ilyen nyelveket elfogadó, a tartalmazás kérdésére választ adó automataosztály a véges automaták osztálya. Ezek a legegyszerűbb automaták. Csak olvasni tudnak, tehát modellezésüknél lyukszalag olvasót képzelhetünk el, és nincs memóriája.

A=<K, V, δ, q0, F>
K: automata belső állapotainak véges halmaza, véges, nem üres halmaz
V: elemzendő jelsorozat abc-je, „szalagabc”
δ: automata mozgási szabályainak véges halmaza,
q_0: induló állapot, $q_0 \in K$
F: végállapotok halmaza, $F \in K$

Egy automata L(M) nyelve alatt azon jelsorozatok halmazát értjük, amelyeket az automata elfogad.
A véges automatákat egy gráffal szemléltethetjük. A gráf irányított, és csomópontjai megfelelnek az automata állapotainak. Amennyiben egy karakter beolvasásának hatására az automata egyik állapotból egy másik állapotba megy át, akkor a két állapotnak megfelelő csomópontokat egy, az eredeti állapotból az új állapotba mutató, és az olvasott karaktert mint nevet viselő éllel kötjük össze. Például δ(A,a)=B: az A csomópontból a B csomópontba „a” jelű él vezet.



Minden véges automatához rendelhetünk egy reguláris nyelvtant, és minden reguláris nyelvtanhoz egy véges automatát. Az automata minden mozgási szabályához rendelhető egy levezetési szabály, így az automata által elfogadott nyelv a nyelvtan által generált nyelv lesz: $\delta(A, a)=B$; $A \rightarrow aB$

Ügyfél-kiszolgáló modell (Kliens-szerver modell): két számítógépes program közötti kapcsolatot ír le, ahol az egyik program valamilyen szolgáltatást kér a másiktól, amely eleget tesz a kérésnek. A szolgáltatást kérő program a kliens, a szolgáltatást nyújtó program a szerver. A modell szerepe főként hálózati környezetben jelentős, ahol a programok egymástól fizikailag is távol, különböző számítógépeken futnak.

Ügyfél-Kiszolgáló felépítmény megvalósítása:
- Ügyfél-kiszolgáló összekapcsolás, mint OO kapcsolat (association)
- Többszörösségek: hány ügyfél használhat egy kiszolgálót
• ügyfél/0..* - a kiszolgáló folyamatosan üzemel
• ügyfél/1..* - a kiszolgálót az első ügyfél indítja, az utolsó ügyfél lekapcsolja
- Tartalmazási kapcsolat/élettartam: kliensenkénti kiszolgálópéldány
- Multi-threading/többszálúság: ügyfél szolgáltatásai párhuzamosan is hívhatók/egyidejűleg több szolgáltatás is futhat

1.A kiszolgáló egypéldányú osztály, a szolgáltatások egymástól függetlenek → állapot nélküli (stateless) kiszolgáló
- ELŐNY: 1. a szerver összeomlása esetén bármikor, minden további nélkül újra indítható
  2. a szolgáltatások nagyobb szabású műveletek/tranzakciók
- HÁTRÁNY: állapotinformációkat minden híváskor el kell küldeni

2. A kiszolgáló több osztályt is rendelkezésre bocsát, a szolgáltatások nem függetlenek egymástól → többállapotú (stateful) kiszolgáló
- ELŐNY: OO környezetben könnyebben szétoszthatóa kiszolgáló több összetevőre
- HÁTRÁNY: 1. könnyen túl aprózódottá válhatnak a hívások és tranzakciók → megterheli a hálózatot
  2. összeomlás utáni visszaállíthatóság problémás

Réteges szoftverfelépítmények: az egyes rétegek elvonatkoztatási szinteket és fizikai rétegeket jelentenek, azaz hardvert és szoftvert is. A hasonló szinten lévő műveletek, szolgáltatások kerülnek azonos környezetbe. A rétegek egyrészt függőleges tagoltságot adnak, másrészt a rétegeken belüli megosztottság vízszintes tagoltságot is biztosít. Minden egyes réteg egymástól független összetevőkből áll, minden egyes réteg szolgáltatásai csak a felette lévő rétegből használhatók. Jellemzően három nagyobb egységre oszthatóak fel:
- Front-end, előtér: GUI (grafikus kezelőfelület)
- Middleware, középréteg: Üzleti logika (BOM, Business Object Model))
  (kétrétegű felépítménynél ez részben/egészében lehet az ügyfélnél, háromrétegűnél elkülönül)
- Back-end, háttér: Adatbázis

COM (Component Object Model): a Microsoft által kifejlesztett technológia a komponens alapú fejlesztés támogatására, mely a szoftverek közti kommunikációt teszi lehetővé. A különböző komponens típusok osztályazonosítókkal azonosíthatóak, melyek globálisan egyéni azonosítók. Minden COM komponens egy vagy több interfészen keresztül szolgáltat funkciókat. A különböző, komponens által támogatott interfészek interfész azonosítókkal különböztethetőek meg egymástól. A COM interfészeknek több programozási nyelvhez van kötése, például C, C++, Visual Basic, valamint több Windows platformon implementált script nyelvhez. A hozzáférés a komponensekhez az interfészek metódusain keresztül történik. Ez lehetővé teszi a folyamatközi/számítógépközi programozást.
A COM programozók felelősek a COM környezetbe való belépésért és az onnan való kilépésért, a COM objektumok inicializálásáért és referenciaszámlálásáért, az objektumok verzióinformációjának lekérdezéséért, a fejlettebb objektum verziók lehetőségeit kihasználó kódért és a visszalépésért az újabb verzió hiányában. A COM osztályokat Globálisan egyedi azonosítók (GUID) azonosítják. A COM osztályokat regisztrálni kell a Regisztrációs adatbázisban. COM objektumokra folyamaton belül, folyamatok között és hálózaton keresztül lehet hivatkozni. A COM objektum figyeli a saját referenciaszámlálóját, és felszabadítja önmagát, ha a számláló értéke nulla.